Ain Shams University
FACULTY OF ENGINEERING
DEPARTMENT OF COMPUTER AND AI ENGINEERING

MILESTONE 1: OOP FOUNDATIONS

HOTEL RESERVATION SYSTEM

Desktop Application in Java

Course: Object Oriented Programming (CSE142)
Team Number: 35

Student Name	Student ID	Section
Ali Mahmoud Diaa El Deen Mohamed	25P0405	10
Abdelrahman Tamer Hamdy Ahmed Ragheb	25P0213	10
Ahmed Eslam Ahmed	25P0401	10
Abdelraouf Mohamed Abdelraouf	25P0439	10
Mahmoud Ahmed	25P0387	10

Contents

1 Introduction **3**
1.1 Project Overview . . . 3
1.2 Objectives of the System . . . 3
1.3 Scope of the Project . . . 3
1.4 Methodology . . . 3
1.5 Significance of the Project . . . 4
1.6 Team Contributions . . . 4
1.6.1 Architectural Highlights . . . 5

2 The Hotel Core Structure **6**
2.1 Overview . . . 6
2.2 Room Class . . . 6
2.2.1 UML Diagram . . . 6
2.2.2 Class Description . . . 6
2.2.3 Validation and Exception Handling . . . 6
2.2.4 Relationship with Database . . . 7
2.2.5 Methodology and Logic Flow . . . 7
2.3 RoomType Class . . . 8
2.3.1 UML Diagram . . . 8
2.3.2 Methodology . . . 8
2.4 Amenity Class . . . 8
2.4.1 UML Diagram . . . 8
2.4.2 Methodology . . . 9
2.5 Architectural Summary . . . 9

3 Booking System **10**
3.1 Overview . . . 10
3.2 System Architecture . . . 10
3.3 Reservation Class . . . 10
3.3.1 Role of the Class . . . 10
3.3.2 UML Diagram . . . 11
3.3.3 Flowchart: Reservation Creation Process . . . 12
3.4 Invoice Class . . . 12
3.4.1 UML Diagram . . . 13
3.4.2 Financial Logic Flow . . . 13
3.5 Payable Interface . . . 14
3.5.1 Purpose of the Interface . . . 14
3.6 Booking System Architecture . . . 14
3.7 Sequence Diagram . . . 14
3.8 Summary . . . 14

4 User Management System **15**
4.1 Overview 15
4.2 Guest Class 15
4.2.1 Role of the Class 15
4.2.2 UML Diagram 15
4.2.3 Validation Protocol 16
4.3 The Staff Hierarchy (Abstract Architecture) 16
4.3.1 The Staff Abstract Class 16
4.3.2 UML Diagram 16
4.3.3 Specialized Actors 17
4.4 Operational Workflows 17
4.4.1 Check-In/Check-Out Process 17
4.4.2 Administrative Room Management 18

5 System Design & Hierarchy **19**
5.1 Inheritance and Encapsulation 19
5.2 Global Class Relationship Overview 19
5.3 Exception Handling Strategy 20
5.4 Summary of the Project 20

Chapter 1

INTRODUCTION

1.1 PROJECT OVERVIEW

The rapid advancement of software systems has significantly transformed the hospitality industry, enabling hotels to manage operations more efficiently and deliver improved customer experiences. This project presents the design and implementation of a **Desktop Hotel Reservation System** developed using the Java programming language, with a strong emphasis on Object-Oriented Programming (OOP) principles.

1.2 OBJECTIVES OF THE SYSTEM

The primary objective of this project is to develop a modular and scalable desktop application that efficiently manages hotel reservations. The system aims to:

- Provide a structured approach to managing rooms, including categorization and amenities.
- Enable seamless reservation creation, validation, and lifecycle management.
- Implement a financial module to calculate booking costs and handle payment methods.
- Support multiple user roles with varying permissions.
- Demonstrate the effective use of OOP concepts including encapsulation, inheritance, polymorphism, and abstraction.

1.3 SCOPE OF THE PROJECT

This project focuses on building a standalone desktop application that operates using an in-memory data management approach. The system supports core functionalities such as room allocation, booking management, and user interaction without relying on external databases.

1.4 METHODOLOGY

The development of the system follows a structured Object-Oriented design methodology. Each component is modeled as a class with clearly defined responsibilities, ensuring high cohesion and low coupling.

1.5 SIGNIFICANCE OF THE PROJECT

This project serves as a practical demonstration of how Object-Oriented Programming can be applied to develop structured, maintainable, and scalable software systems.

1.6 TEAM CONTRIBUTIONS

The development of the Hotel Management System was a collaborative effort, structured to ensure modularity and high code standards. Each team member was responsible for a specific architectural domain, as detailed in the matrix below:

Team Member	Area of Responsibility
Abdelraouf	**Physical Infrastructure:** Designed the Room and RoomType models, managed the Amenities logic, and defined the associated system Enums.
Mahmoud	**Financial Engine:** Architected the Booking system, including the Reservation lifecycle, Invoice generation, and the Payable interface contract.
Ali	**User Hierarchy:** Implemented the Abstract Staff architecture, Administrative management (Manageable interface), and Receptionist operations.
Ahmed	**System Integrity:** Engineered the global Exception handling framework and enforced strict data validation protocols across all classes.
Abdelrahman	**Data Services:** Managed the Guest entity and implemented external services, including the SQL DatabaseManager and the CSVExportService for periodic Excel reporting.
Joint Effort	**System Integration:** Collaborative development of the Main execution entry point and cross-module synchronization.

Table 1.1: Team Responsibility and Domain Distribution

1.6.1 ARCHITECTURAL HIGHLIGHTS

The division of labor was designed to leverage specific Object-Oriented principles:

- **Persistence Layer:** Handled by the `DatabaseManager` to ensure the application works with SQL-based real data.
- **Reporting:** The `CSVExportService` provides periodic Guest Information reports, bridging the gap between Java objects and Excel-compatible data.
- **Safety:** A centralized focus on `Exceptions` ensured that the application handles runtime errors gracefully without crashing.

Chapter 2

THE HOTEL CORE STRUCTURE

2.1 OVERVIEW

The Hotel Core Structure represents the foundational layer of the system.

- Each **Room** is associated with exactly one **RoomType**.
- Each **Room** contains multiple **Amenity** objects using aggregation.
- Pricing and capacity logic are delegated to **RoomType**.

2.2 ROOM CLASS

2.2.1 UML DIAGRAM

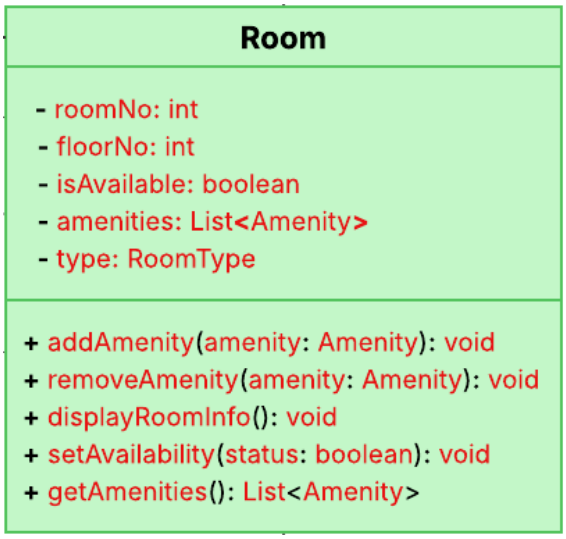

Figure 2.1: UML Diagram of Room Class.

2.2.2 CLASS DESCRIPTION

The Room class represents a physical hotel room and acts as the central entity. It encapsulates identification attributes, classification, and availability state.

2.2.3 VALIDATION AND EXCEPTION HANDLING

- **InvalidRoomDataException:** For negative numbers or null RoomType.
- **DuplicateAmenityException:** When adding an existing amenity name.
- **AmenityNotFoundException:** For invalid removals.

2.2.4 RELATIONSHIP WITH DATABASE

Stored in `HotelDatabase.rooms`, representing a centralized in-memory persistence model initialized at startup.

2.2.5 METHODOLOGY AND LOGIC FLOW

addAmenity(Amenity amenity) Flowchart

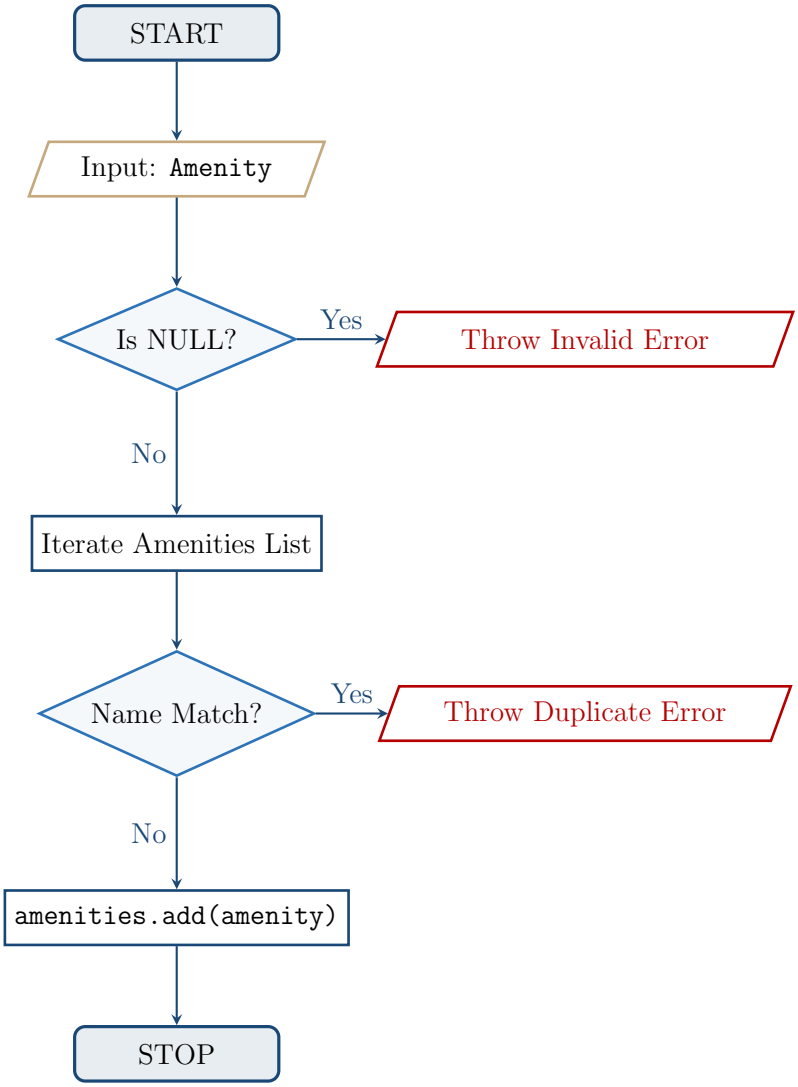

Figure 2.2: Professional Logical Flow for Room Amenity Validation.

2.3 ROOMTYPE CLASS

2.3.1 UML DIAGRAM

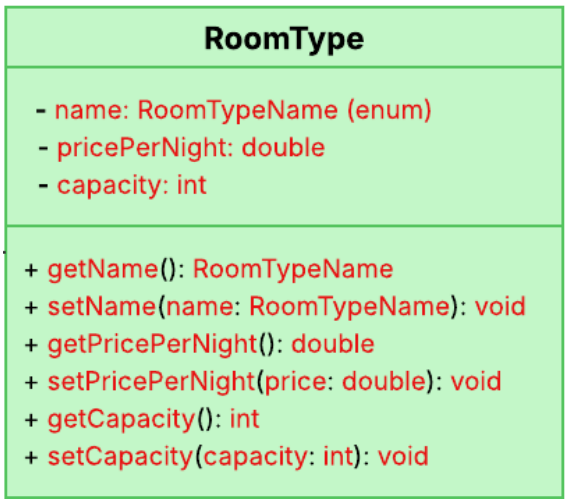

Figure 2.3: UML Diagram of Room Class.

2.3.2 METHODOLOGY

Constructor Logic: Calls setters to enforce validation.
Setters: Ensures name is not null, price ≥ 0, and capacity > 0.

2.4 AMENITY CLASS

2.4.1 UML DIAGRAM

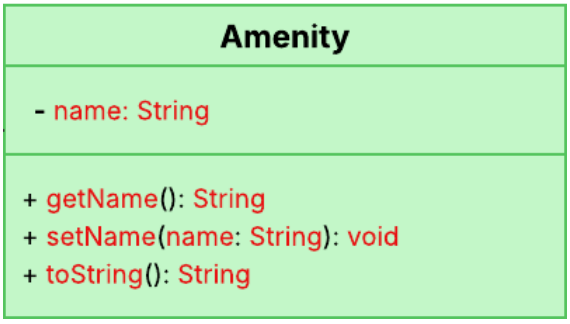

Figure 2.4: UML Diagram of Room Class.

2.4.2 METHODOLOGY

Constructor Logic: Calls `setName()` to enforce validation.
setName(): Prevents null/empty/blank names and trims whitespace.

2.5 ARCHITECTURAL SUMMARY

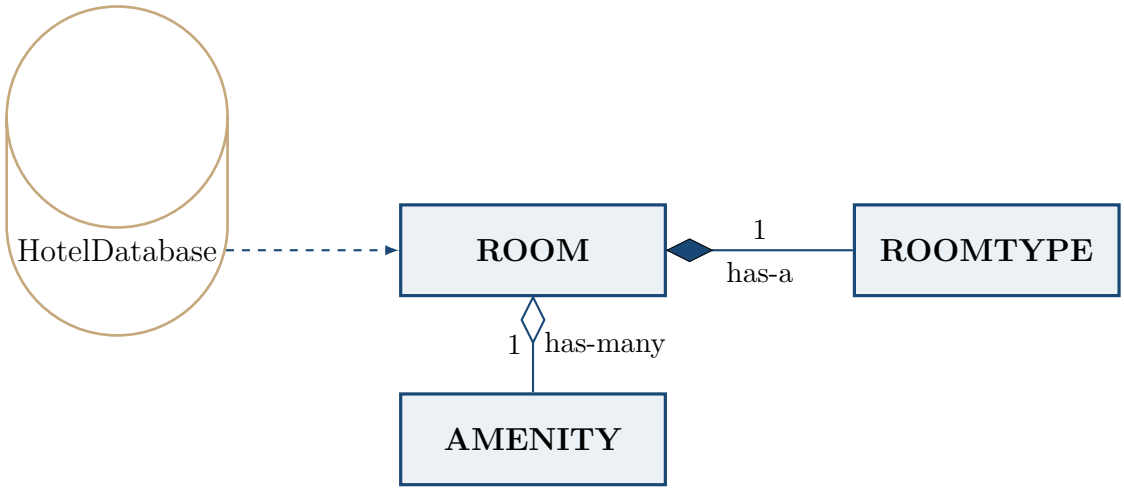

Figure 2.5: Relationship Between Room, RoomType, and Amenity within the Persistence Layer.

Chapter 3

BOOKING SYSTEM

3.1 OVERVIEW

The Booking System is the core functional module of the hotel reservation application. It is responsible for managing the full lifecycle of a reservation, starting from the moment a guest requests a room until the final payment is completed and the booking is closed.

This system integrates three major components:

- **RESERVATION MANAGEMENT** — Handles booking creation, validation, and status tracking.
- **INVOICE SYSTEM** — Handles financial calculations and payment processing.
- **PAYABLE INTERFACE** — Defines a contract for all financial entities in the system.

The design follows Object-Oriented Programming principles to ensure scalability, modularity, and clear separation of responsibilities.

3.2 SYSTEM ARCHITECTURE

The Booking System is built around strong relationships between entities:

- A **Guest** can create multiple **Reservations**.
- Each **Reservation** is linked to exactly one **Room**.
- Each **Reservation** generates exactly one **Invoice**.
- The **Invoice** implements the **Payable interface**.

3.3 RESERVATION CLASS

3.3.1 ROLE OF THE CLASS

The Reservation class represents the core booking entity. It acts as a bridge between the Guest and the Room while controlling booking dates, status (PENDING, CONFIRMED, CANCELLED), and enforcing validation rules to ensure system consistency.

3.3.2 UML DIAGRAM

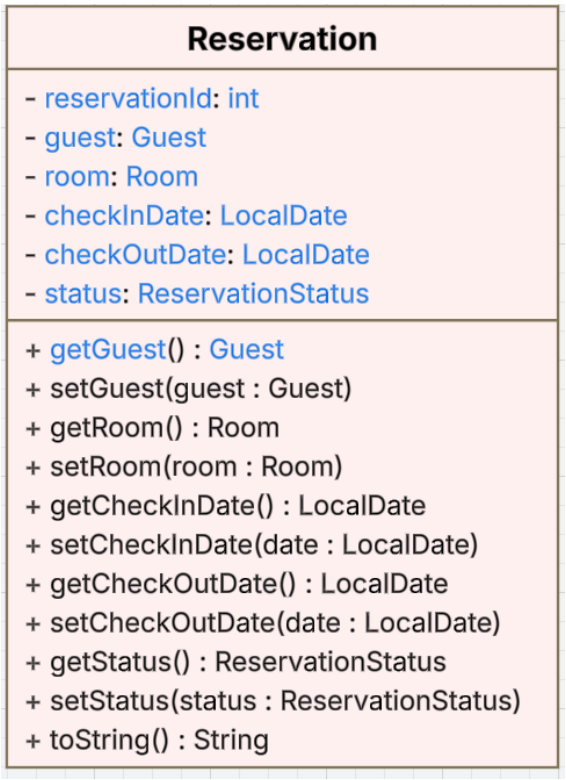

Figure 3.1: UML Diagram of Reservation Class

3.3.3 FLOWCHART: RESERVATION CREATION PROCESS

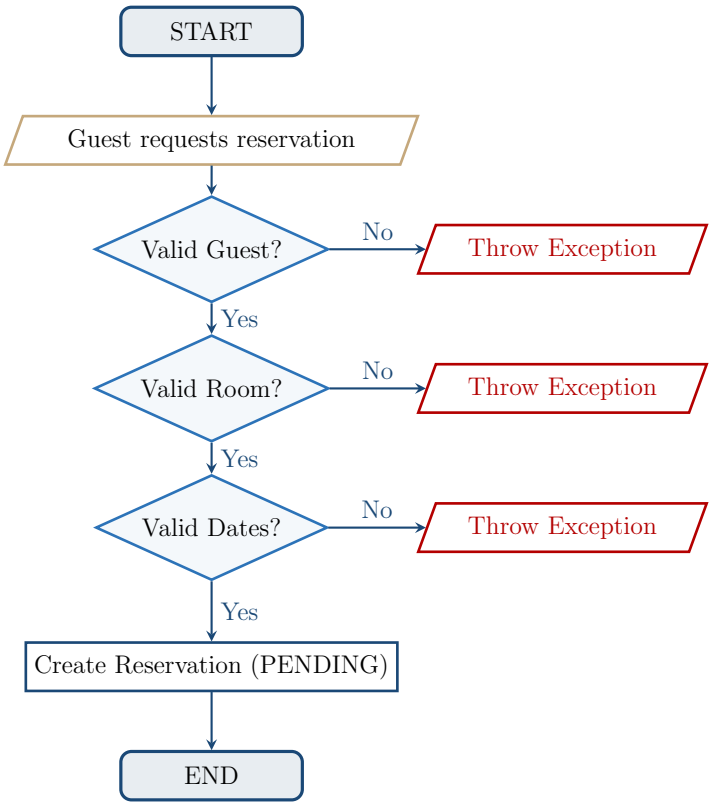

Figure 3.2: Reservation Creation Flowchart

3.4 INVOICE CLASS

3.4.1 UML DIAGRAM

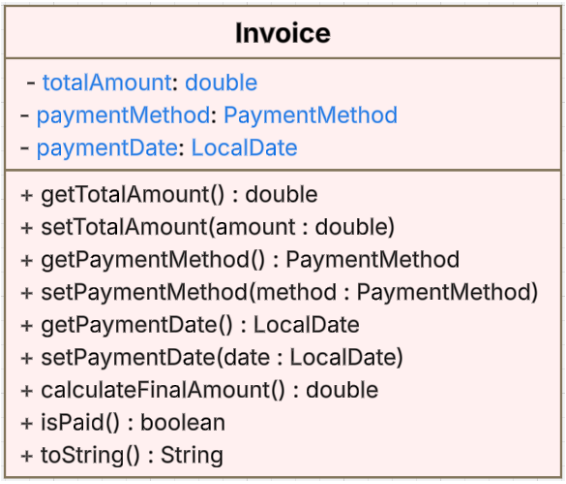

Figure 3.3: UML Diagram of Invoice Class

3.4.2 FINANCIAL LOGIC FLOW

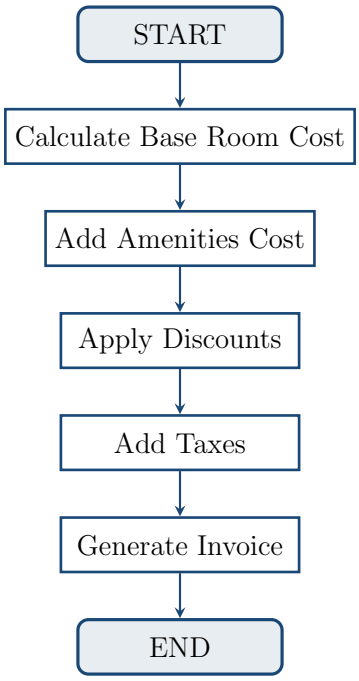

Figure 3.4: Invoice Calculation Flow

3.5 PAYABLE INTERFACE

3.5.1 PURPOSE OF THE INTERFACE

The Payable interface defines a contract for all financial entities. It ensures that any class implementing it must provide `calculateFinalAmount()` and `isPaid()`, introducing **polymorphism** for multiple financial models.

3.6 BOOKING SYSTEM ARCHITECTURE

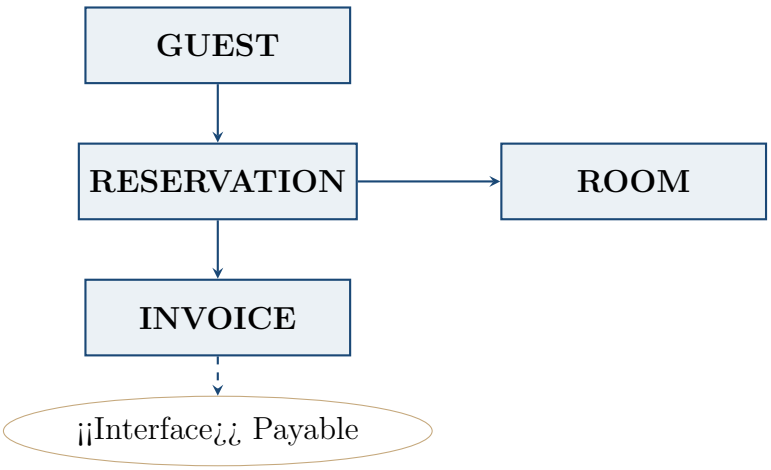

Figure 3.5: Booking System Structural Overview

3.7 SEQUENCE DIAGRAM

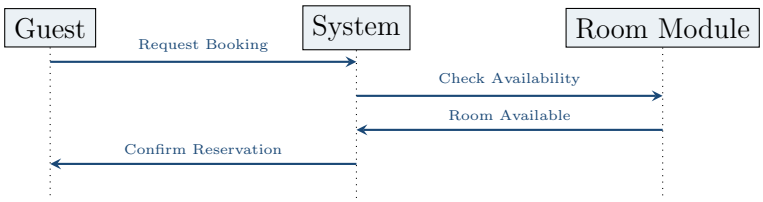

Figure 3.6: Sequence Diagram of Booking Process

3.8 SUMMARY

The Booking System ensures reliable lifecycle management of reservations by separating booking logic from financial calculations, maintaining data integrity through strict validation rules.

Chapter 4

USER MANAGEMENT SYSTEM

4.1 OVERVIEW

The User Management System defines the actors within the Hotel Application. It utilizes an inheritance-based architecture to separate the concerns of the service providers (**Staff**) from the service consumers (**Guests**). By using an abstract base class for staff, the system enforces a common security and data protocol while allowing for specialized behaviors like administrative management and receptionist operations.

4.2 GUEST CLASS

4.2.1 ROLE OF THE CLASS

The **Guest** class is a data-centric entity representing the hotel's clientele. Unlike Staff, Guests do not have system-wide authority but carry personal attributes, address data, and specific stay preferences (e.g., Room Type Preference).

4.2.2 UML DIAGRAM

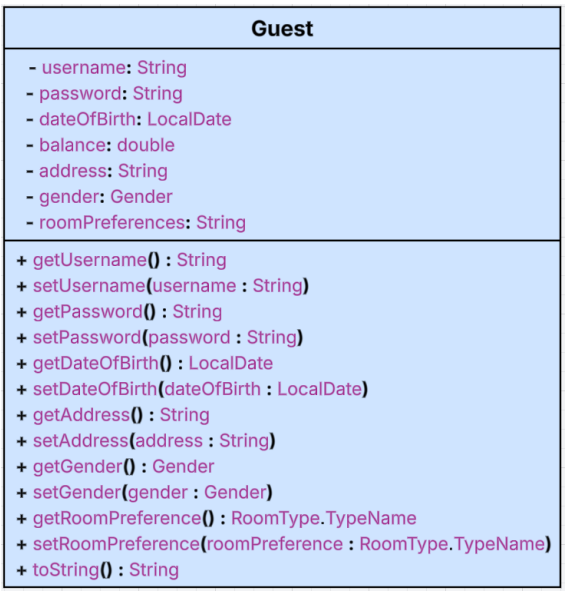

Figure 4.1: UML Diagram of Guest Entity

4.2.3 VALIDATION PROTOCOL

To maintain a high-quality database, the Guest class implements the following constraints:

- **Identity Validation:** Username and Password cannot be null or empty strings.
- **Chronological Check:** Date of Birth must be a valid date and cannot be set in the future.
- **Preference Integrity:** Every guest must have an assigned Gender and Room Preference from the system enums.

4.3 THE STAFF HIERARCHY (ABSTRACT ARCHITECTURE)

4.3.1 THE STAFF ABSTRACT CLASS

The **Staff** class serves as the blueprint for all employees. As an **Abstract Class**, it cannot be instantiated directly. It centralizes shared attributes like `workingHours` and `Role`, and provides shared behaviors such as viewing the global list of Guests and Rooms.

4.3.2 UML DIAGRAM

Figure 4.2: UML Diagram of Staff Abstract Hierarchy

4.3.3 SPECIALIZED ACTORS

The system branches into two distinct operational roles:

1. Admin (Management Actor)

The **Admin** is the most powerful actor in the system. Beyond standard staff views, the Admin implements the **Manageable** interface, allowing for full CRUD (Create, Read, Update, Delete) operations on the hotel's room inventory.

2. Receptionist (Operational Actor)

The **Receptionist** manages the daily lifecycle of a reservation. They are responsible for the physical transition of a room from "Available" to "Occupied" during the check-in process.

4.4 OPERATIONAL WORKFLOWS

4.4.1 CHECK-IN/CHECK-OUT PROCESS

The Receptionist interacts with the **Room** and **Reservation** classes to update the system state in real-time.

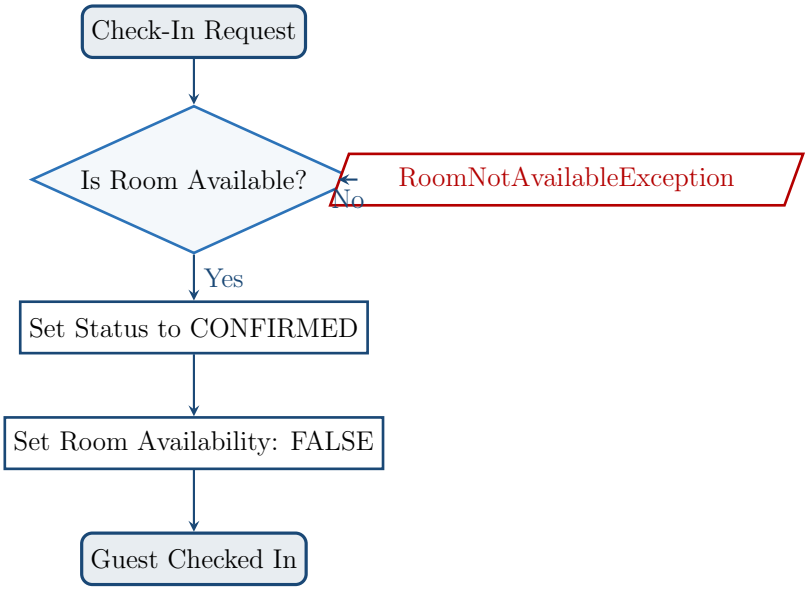

Figure 4.3: Receptionist Check-In Workflow

4.4.2 ADMINISTRATIVE ROOM MANAGEMENT

The Admin manages the `ArrayList<Room>` using a robust search-and-modify pattern.

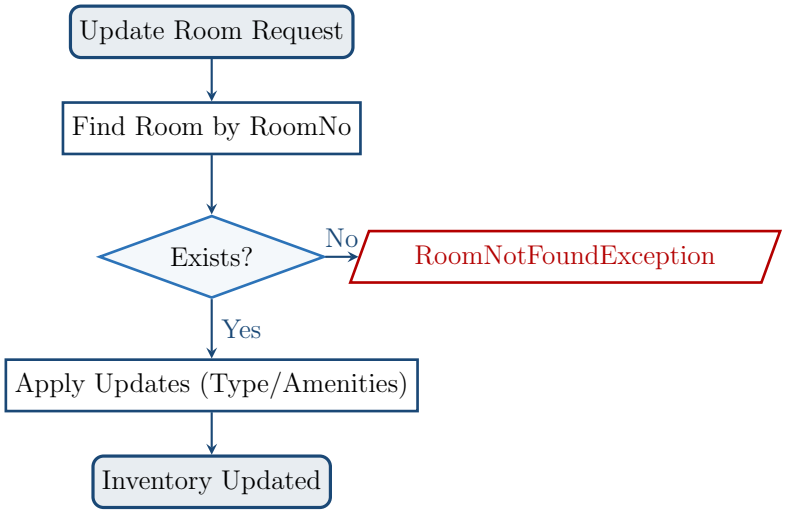

Figure 4.4: Administrative CRUD Workflow

Chapter 5

SYSTEM DESIGN & HIERARCHY

5.1 INHERITANCE AND ENCAPSULATION

This project heavily relies on the four pillars of Object-Oriented Programming:

- **Encapsulation:** All fields in `Guest`, `Staff`, and `Reservation` are private. Access is strictly controlled through public setters that contain validation logic.
- **Inheritance:** The `Admin` and `Receptionist` classes inherit from `Staff`, reducing code duplication for shared attributes like `username` and `workingHours`.
- **Abstraction:** The `Staff` class and the `Payable` interface provide a high-level template without dictating specific implementation details.
- **Polymorphism:** The `Manageable` and `Payable` interfaces allow the system to treat different objects through a common interface.

5.2 GLOBAL CLASS RELATIONSHIP OVERVIEW

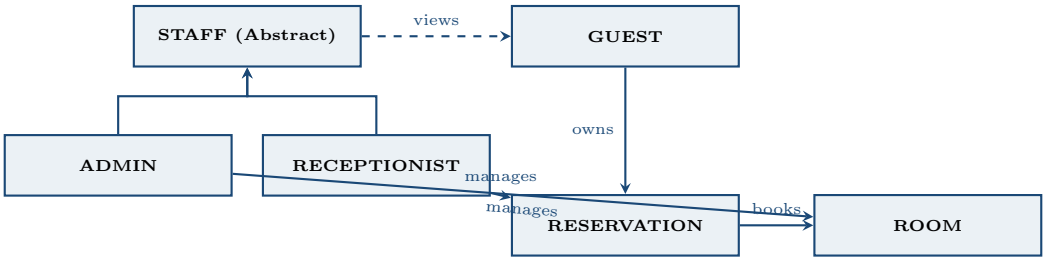

Figure 5.1: Global System Architecture and Actor Dependencies

5.3 EXCEPTION HANDLING STRATEGY

The system uses a custom exception hierarchy to ensure that logical errors (like booking a room that is already taken) do not result in system crashes.

Exception Name	Responsibility
`RoomNotFoundException`	Triggered when an Admin tries to modify a non-existent ID.
`RoomNotAvailableException`	Triggered if a Receptionist checks a guest into an occupied room.
`InvalidGuestDataException`	Triggered when a Guest profile violates age or name rules.
`DuplicateRoomException`	Prevents the creation of two rooms with the same number.

Table 5.1: System-wide Safety and Exception Management

5.4 SUMMARY OF THE PROJECT

The Hotel Management System provides a robust, scalable framework for digital hospitality management. By separating the technical infrastructure (Rooms), the financial engine (Invoices), and the human actors (Staff/Guests), the application ensures high maintainability and a professional user experience.

The implementation of strict validation in the **Reservation** class and the flexible management tools in the **Admin** class creates a system that is both secure and efficient, capable of handling the complexities of modern hotel operations.